EXPERIMENT NO: 2

AIM - **Experiments of Packet capture tool: Wireshark**

To understand the features of wireshark as a packet capture tool and understand encapsulation of information. Also study effect of few network commands

Purpose:

- To **capture and analyze packets** flowing through a network interface.
- To **study encapsulation**: how headers are added at each layer (Data Link, Network, Transport, Application).
- To **observe protocol behavior** (e.g., TCP handshake, DNS queries, ICMP echo).
- To **verify network commands** by seeing their actual packet traces.
- To **learn troubleshooting skills** for latency, dropped packets, or misconfigurations.

Description

- **Wireshark** is the world's most widely used **network protocol analyzer**.
- It can capture live traffic from Ethernet, Wi-Fi, Bluetooth, and more.
- Provides a **graphical interface** with detailed protocol breakdowns.
- Supports **hundreds of protocols** (HTTP, DNS, TCP, UDP, TLS, ARP, ICMP, etc.).
- Useful for **students, administrators, and security professionals** to understand how networks actually work.
- Open-source and free, with a large community and documentation support.

Downloading Wireshark

- Visit wireshark.org.

1. Download the Windows installer (.exe).
2. Run the installer → keep default settings.
3. Install **Npcap** when prompted (required for packet capture).
4. Finish installation and launch Wireshark.
5. Select your active interface (Wi-Fi/Ethernet).
6. Click the **blue shark icon** to start capture.

Packet Capturing

- Wireshark places NIC in promiscuous mode to capture all packets.
- Captures are stored in .pcap or .pcapng files.
- Supports capture filters (before capture) and display filters (after capture).

Examples of Capture Filters:

- host 192.168.1.10 → only traffic to/from this host.
- net 10.0.0.0/24 → only traffic in this subnet.
- src port 53 → only DNS queries.
- tcp and not port 80 → all TCP except HTTP.

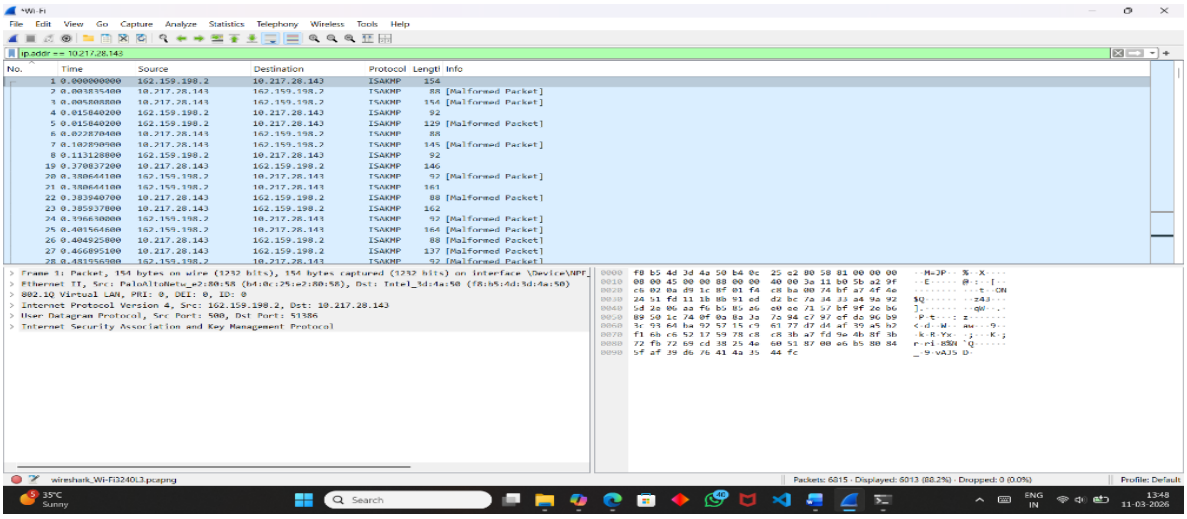
Analysis Views

Wireshark provides three panes:

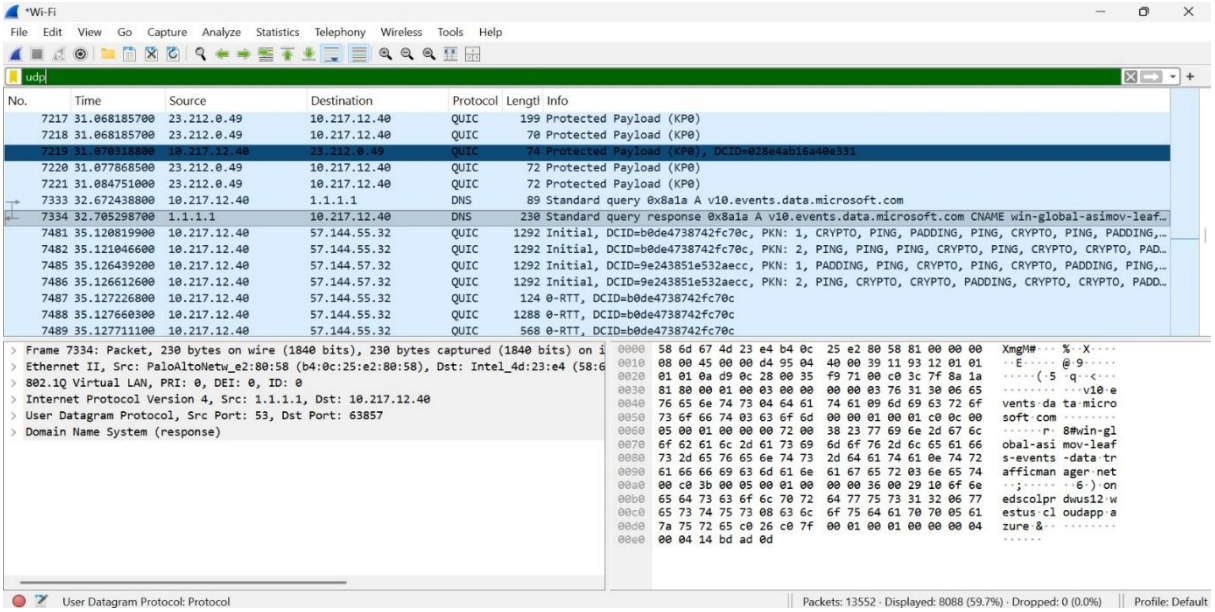
1. Packet List Pane → summary of packets.
2. Packet Details Pane → layer-by-layer breakdown (Ethernet → IP → TCP → HTTP).
3. Packet Bytes Pane → raw hexadecimal + ASCII data.

Outputs:

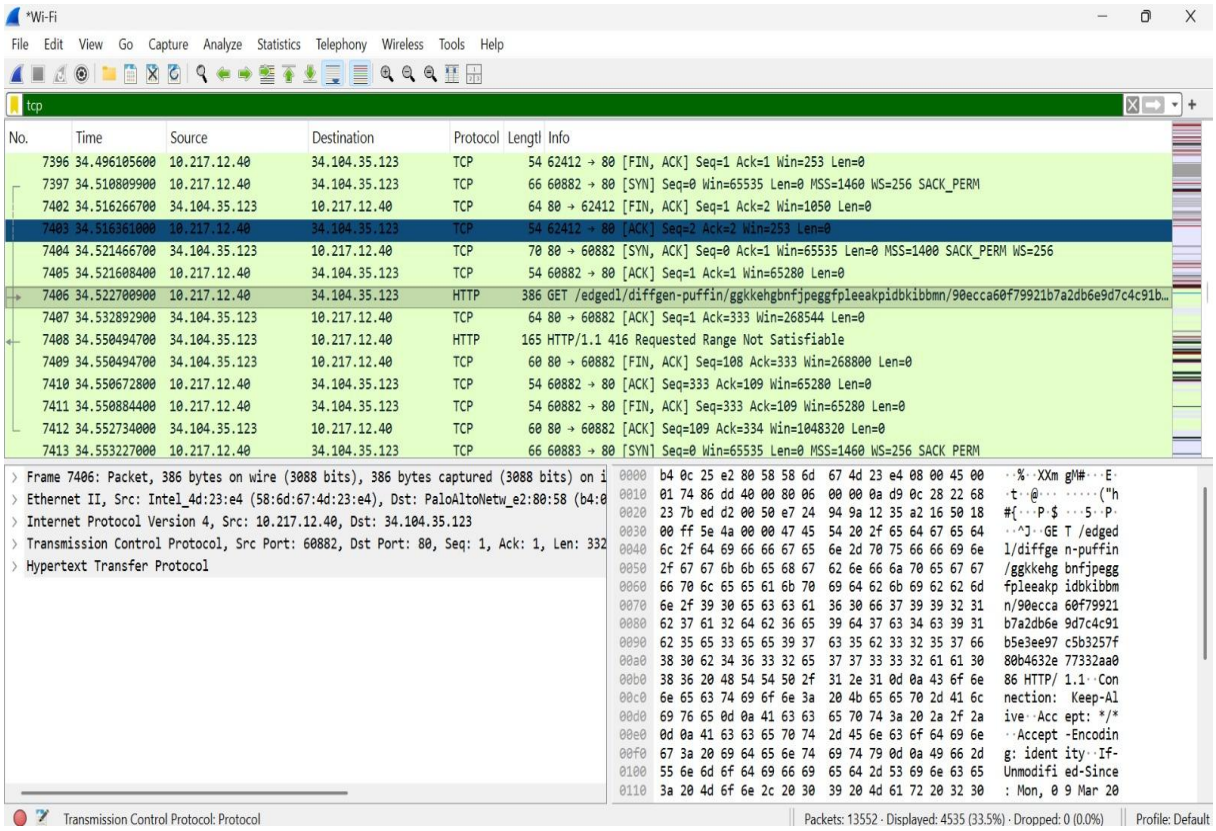
- **ip.addr == 10.217.28.143**



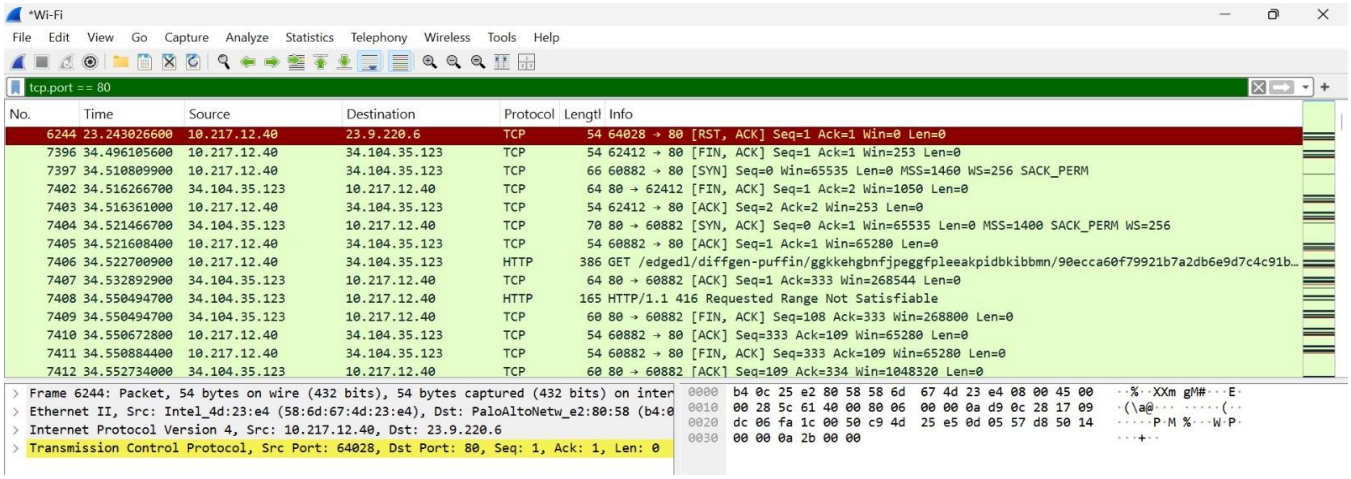
- **udp**


*Wi-Fi
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
udp
No. Time Source Destination Protocol Lengtl Info
7217 31.068185700 23.212.0.49 10.217.12.40 QUIC 199 Protected Payload (KP0)
7218 31.068185700 23.212.0.49 10.217.12.40 QUIC 70 Protected Payload (KP0)
7219 31.070318800 10.217.12.40 23.212.0.49 QUIC 74 Protected Payload (KP0), DCID=028e4ab16a40e331
7220 31.077868500 23.212.0.49 10.217.12.40 QUIC 72 Protected Payload (KP0)
7221 31.084751000 23.212.0.49 10.217.12.40 QUIC 72 Protected Payload (KP0)
7333 32.672438800 10.217.12.40 1.1.1.1 DNS 89 Standard query 0x8a1a A v10.events.data.microsoft.com
7334 32.705298700 1.1.1.1 10.217.12.40 DNS 230 Standard query response 0x8a1a A v10.events.data.microsoft.com CNAME win-global-asimov-leaf…
7481 35.120819900 10.217.12.40 57.144.55.32 QUIC 1292 Initial, DCID=b0de4738742fc70c, PKN: 1, CRYPTO, PING, PADDING, PING, CRYPTO, PING, PADDING,…
7482 35.121046600 10.217.12.40 57.144.55.32 QUIC 1292 Initial, DCID=b0de4738742fc70c, PKN: 2, PING, PING, PING, CRYPTO, PING, CRYPTO, CRYPTO, PAD…
7485 35.126439200 10.217.12.40 57.144.57.32 QUIC 1292 Initial, DCID=9e243851e532aecc, PKN: 1, PADDING, PING, CRYPTO, PING, CRYPTO, PADDING, PING,…
7486 35.126612600 10.217.12.40 57.144.57.32 QUIC 1292 Initial, DCID=9e243851e532aecc, PKN: 2, PING, CRYPTO, CRYPTO, PADDING, CRYPTO, CRYPTO, PADD…
7487 35.127226800 10.217.12.40 57.144.55.32 QUIC 124 0-RTT, DCID=b0de4738742fc70c
7488 35.127660300 10.217.12.40 57.144.55.32 QUIC 1288 0-RTT, DCID=b0de4738742fc70c
7489 35.127711100 10.217.12.40 57.144.55.32 QUIC 568 0-RTT, DCID=b0de4738742fc70c
Frame 7334: Packet, 230 bytes on wire (1840 bits), 230 bytes captured (1840 bits) on i
Ethernet II, Src: PaloAltoNetw_e2:80:58 (b4:0c:25:e2:80:58), Dst: Intel_4d:23:e4 (58:6
802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 0
Internet Protocol Version 4, Src: 1.1.1.1, Dst: 10.217.12.40
User Datagram Protocol, Src Port: 53, Dst Port: 63857
Domain Name System (response)
User Datagram Protocol: Protocol
Packets: 13552 · Displayed: 8088 (59.7%) · Dropped: 0 (0.0%)
Profile: Default

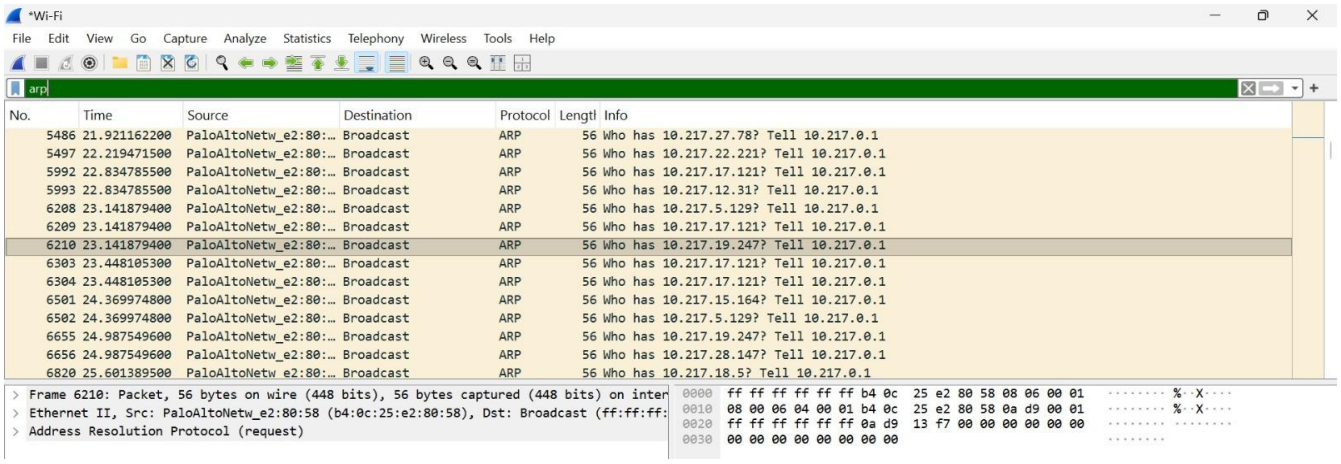
tcp


*Wi-Fi
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
tcp
No. Time Source Destination Protocol Lengtl Info
7396 34.496105600 10.217.12.40 34.104.35.123 TCP 54 62412 → 80 [FIN, ACK] Seq=1 Ack=1 Win=253 Len=0
7397 34.510809900 10.217.12.40 34.104.35.123 TCP 66 60882 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
7402 34.516266700 34.104.35.123 10.217.12.40 TCP 64 80 → 62412 [FIN, ACK] Seq=1 Ack=2 Win=1050 Len=0
7403 34.516361000 10.217.12.40 34.104.35.123 TCP 54 62412 → 80 [ACK] Seq=2 Ack=2 Win=253 Len=0
7404 34.521466700 34.104.35.123 10.217.12.40 TCP 70 80 → 60882 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1400 SACK_PERM WS=256
7405 34.521608400 10.217.12.40 34.104.35.123 TCP 54 60882 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
7406 34.522700900 10.217.12.40 34.104.35.123 HTTP 386 GET /edged1/diffgen-puffin/ggkkehgbnfjpeggfpleeakpidbkibbmn/90ecca60f79921b7a2db6e9d7c4c91b…
7407 34.532892900 34.104.35.123 10.217.12.40 TCP 64 80 → 60882 [ACK] Seq=1 Ack=333 Win=268544 Len=0
7408 34.550494700 34.104.35.123 10.217.12.40 HTTP 165 HTTP/1.1 416 Requested Range Not Satisfiable
7409 34.550494700 34.104.35.123 10.217.12.40 TCP 60 80 → 60882 [FIN, ACK] Seq=108 Ack=333 Win=268800 Len=0
7410 34.550672800 10.217.12.40 34.104.35.123 TCP 54 60882 → 80 [ACK] Seq=333 Ack=109 Win=65280 Len=0
7411 34.550884400 10.217.12.40 34.104.35.123 TCP 54 60882 → 80 [FIN, ACK] Seq=333 Ack=109 Win=65280 Len=0
7412 34.552734000 34.104.35.123 10.217.12.40 TCP 60 80 → 60882 [ACK] Seq=109 Ack=334 Win=1048320 Len=0
7413 34.553227000 10.217.12.40 34.104.35.123 TCP 66 60883 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
Frame 7406: Packet, 386 bytes on wire (3088 bits), 386 bytes captured (3088 bits) on i
Ethernet II, Src: Intel_4d:23:e4 (58:6d:67:4d:23:e4), Dst: PaloAltoNetw_e2:80:58 (b4:0
Internet Protocol Version 4, Src: 10.217.12.40, Dst: 34.104.35.123
Transmission Control Protocol, Src Port: 60882, Dst Port: 80, Seq: 1, Ack: 1, Len: 332
Hypertext Transfer Protocol
Transmission Control Protocol: Protocol
Packets: 13552 · Displayed: 4535 (33.5%) · Dropped: 0 (0.0%)
Profile: Default

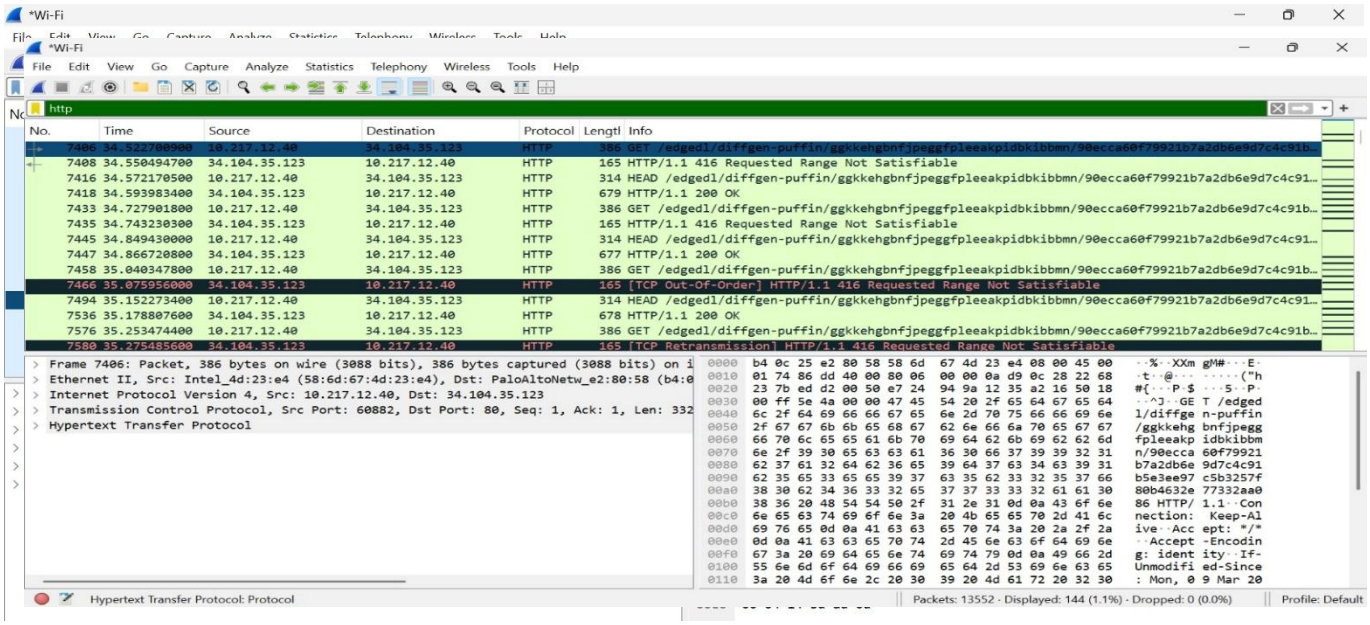
tcp.port == 80



arp

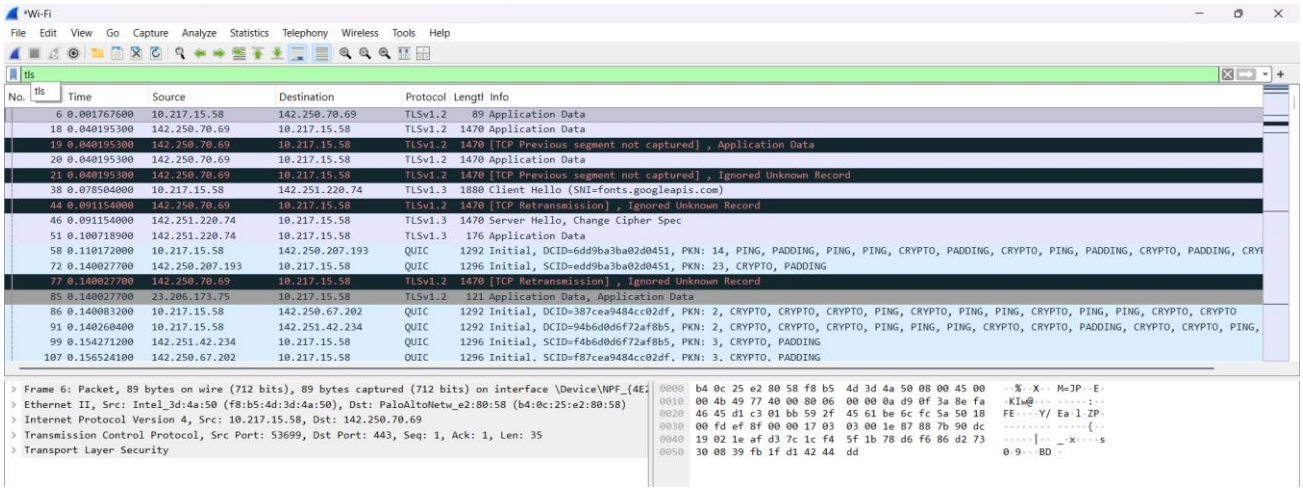


dns

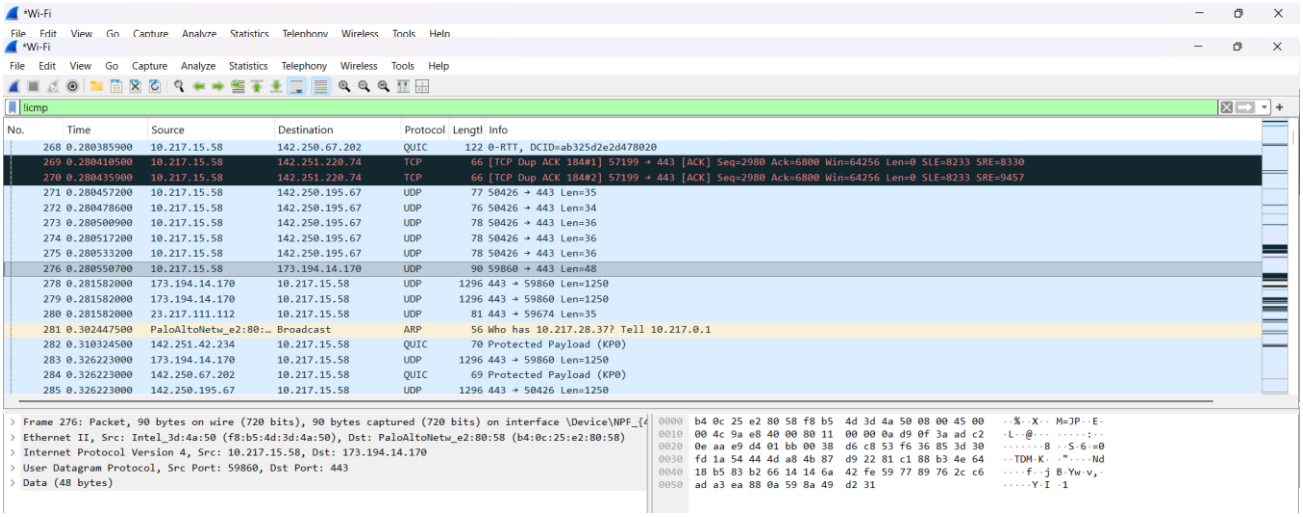


http

tls

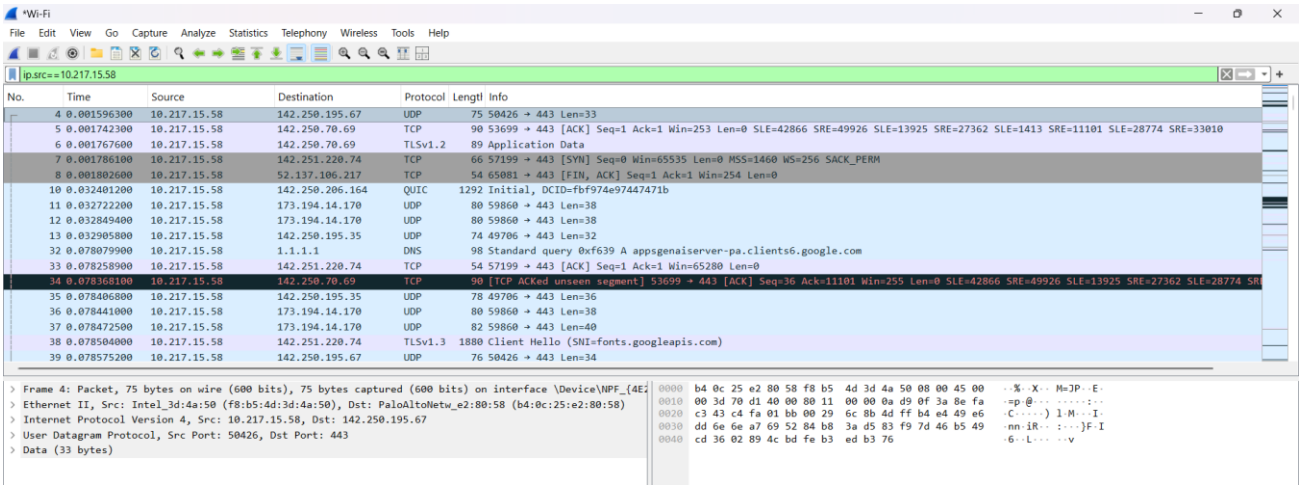


ip.src==10.217.15.58 &&ip.dst ==52.137.106.217



!icmp

ip.src==10.217.15.58



Conclusion:

Wireshark effectively captured and analyzed ICMP, TLS, and FTP traffic, showing how different protocols operate across the OSI layers. ICMP packets confirmed connectivity, TLS highlighted secure encrypted communication, and FTP demonstrated command-based file transfers. This experiment provided clear insight into packet encapsulation and protocol behavior in real network environments.